

INFORMAÇÕES

MÓDULO	AULA	INSTRUTOR
08 ASP.NET Core Web API – Avançado – EF Core, LINQ e Segurança	01 - Entity Framework Core 8 Avançado	Guilherme Paracatu

1. Alta Performance, Operações em Massa e Modelagem Complexa

🎯 Objetivo da Aula

Nesta aula, vamos deixar as operações básicas de CRUD para trás. O objetivo agora é capacitar você a extrair a performance máxima do Entity Framework Core 8 (EF8), lidando com milhares de registros sem travar a API, achatando coleções aninhadas com maestria e utilizando recursos modernos de modelagem de banco de dados, como colunas JSON e Tipos Complexos.

1. Operações em Massa (Bulk Updates & Deletes): O Fim do foreach

O Problema:

Imagine que a diretoria exigiu a aplicação de um desconto de 10% em todos os veículos fabricados antes de 2020. A abordagem ingênua seria buscar todos os registros no banco, trazê-los para a memória (RAM), rodar um laço foreach alterando o preço de um por um, e chamar o `_uow.CommitAsync()`. Se a base tiver 50.000 veículos, sua API vai consumir Gigabytes de memória e disparar 50.000 comandos de UPDATE individuais na rede. Isso pode, e vai, derrubar o servidor.

A Teoria e a Solução de Arquiteto (EF Core 8):

Para resolver isso, o EF8 introduziu os métodos `.ExecuteUpdateAsync()` e `.ExecuteDeleteAsync()`. Mas por que eles são tão rápidos? Esses métodos **ignoram completamente o *Change Tracker*** (o rastreador de entidades na memória do EF). Em vez de carregar os dados no C#, o EF Core pega a sua expressão LINQ e a traduz diretamente para um único comando SQL puro (UPDATE tabela SET ... WHERE ...), que é disparado diretamente no banco de dados.

Entendendo o `.SetProperty`

O método `.ExecuteUpdateAsync()` exige que você diga explicitamente quais colunas serão alteradas através do construtor `.SetProperty()`. Ele funciona recebendo dois parâmetros via expressões lambda:

1. Qual propriedade da entidade você quer alterar.
2. O novo valor (que pode ser um valor fixo ou um cálculo baseado no valor atual da própria coluna, resolvido dentro do banco!).

Exemplo Prático: Aplicando o Desconto em Massa

Código C#

// Dispara apenas 1 comando SQL nativo:

*// UPDATE Veiculos SET Preco = Preco * 0.9, Status = 'Promoção Especial' WHERE Ano < 2020;*

```
var LinhasAfetadas = await _context.Veiculos
    .Where(v => v.Ano < 2020)
    .ExecuteUpdateAsync(setters => setters
        // Altera usando matemática direta no banco:
        .SetProperty(v => v.Preco, v => v.Preco * 0.9m)
        // Altera usando uma string fixa:
        .SetProperty(v => v.Status, "Promoção Especial")
    );
```

Console.WriteLine(\$"Foram atualizados {linhasAfetadas} veículos em milissegundos!");

Regra de Ouro: Operações em massa não passam pela memória e **não precisam do SaveChanges() ou CommitAsync()**. Elas atiram direto no banco!

2. A Mágica do .SelectMany() (O Achatador de Coleções)

O Cenário: Sua API possui uma hierarquia: Marca contém uma lista de Modelos, que por sua vez contém uma lista de Veiculos. O Front-end pediu um relatório linear (uma lista única e contínua) com todos os carros da marca Peugeot. Se você utilizar apenas o .Select(), acabará com uma lista de listas (List<List<Veiculo>>).

A Solução: O operador .SelectMany() serve exatamente para **achatar (flatten)** coleções aninhadas. Ele "mergulha" na lista filha e retorna todos os elementos em um nível único, gerando um INNER JOIN limpo e super otimizado no banco.

Exemplo Prático: Achatando a Hierarquia

Código C#

```
var relatorioFrota = await _context.Marcas
    .Where(m => m.Nome == "Peugeot")
    .SelectMany(marca => marca.Modelos) // Mergulha nos Modelos da Marca
    .SelectMany(modelo => modelo.Veiculos) // Mergulha nos Veículos do Modelo
    .Select(veiculo => new FrotaDTO
    {
        MarcaNome = veiculo.Modelo.Marca.Nome,
        ModeloNome = veiculo.Modelo.Nome,
        Placa = veiculo.Placa,
        Ano = veiculo.Ano
    })
    .ToListAsync();
// Resultado: Uma lista simples e contínua de FrotaDTO pronta para a tela!
```

3. Modelagem Avançada I: Colunas JSON Nativas no EF8

Os bancos de dados relacionais modernos, especialmente o **PostgreSQL**, possuem um suporte incrível para colunas JSON (como o tipo jsonb). Isso permite flexibilidade de banco NoSQL dentro de um banco Relacional.

O Cenário (A Teoria) Queremos guardar o *HistoricoDeManutencao* do veículo (um documento complexo com várias revisões, peças trocadas, quilometragens variadas e datas). Criar tabelas adicionais para isso geraria JOINS pesados e um esquema rígido. A solução é salvar todo esse objeto como um único documento JSONB na tabela Veiculos.

Como Implementar (PostgreSQL e SQL Server): Na sua classe de configuração do EF Core
Código C#

```
// 1. Configurando na Fluent API
builder.Entity<Veiculo>()
    .OwnsOne(v => v.HistoricoManutencao, navigationBuilder =>
    {
        navigationBuilder.ToJson(); // O EF Core criará uma coluna JSONB no
PostgreSQL!
    });
```

O Poder do LINQ no JSON O verdadeiro poder do EF8 é que ele consegue traduzir o seu código C# para buscar dados **dentro** da árvore do JSON no banco:

Código C#

```
// O EF8 traduz esse LINQ para uma busca usando os operadores JSONB nativos
var veiculosRevisados = await _context.Veiculos
    .Where(v => v.HistoricoManutencao.PecasTrocadas.Contains("Correia Dentada"))
    .ToListAsync();
```

4. Modelagem Avançada II: Tipos Complexos (Value Objects)

No *Domain-Driven Design* (DDD), temos o conceito de "Objetos de Valor" (*Value Objects*). São classes que agrupam propriedades que fazem sentido juntas, mas que não possuem identidade (Id) própria. Exemplo: Endereco, Dimensoes, Coordenadas.

A Teoria e a Vantagem Antes do EF8, mapear isso exigia a criação de chaves primárias invisíveis (*shadow properties*). Agora, o EF8 introduziu os Tipos Complexos. Eles instruem o banco a "desmanchar" a classe e criar colunas na própria tabela principal, mantendo o banco relacional rápido e seu código C# muito bem orientado a objetos.

Como Implementar: Basta anotar a classe com [ComplexType].

Código C#

```
// 1. A classe do Objeto de Valor
[ComplexType]
public class Dimensoes
{
    public decimal Altura { get; set; }
```

```

    public decimal Largura { get; set; }
    public decimal Comprimento { get; set; }
}

// 2. Na Entidade Principal
public class Veiculo : Entity
{
    public string Placa { get; set; }
    public Dimensoes DimensoesDoVeiculo { get; set; } // Propriedade agrupada
}

```

No banco de dados, o EF criará na tabela *Veiculos* as colunas: *DimensoesDoVeiculo_Altura*, *DimensoesDoVeiculo_Largura*, etc.

5. Consultas SQL Puras mapeadas para DTOs (SqlQuery<T>)

Quando o LINQ não for suficiente para resolver um gargalo de performance ou você precisar utilizar um recurso exclusivo do banco (como *Hints* ou CTEs recursivas), o EF8 permite executar SQL puro e jogar o resultado direto em uma classe DTO solta, que nem precisa estar registrada no seu DbContext.

Exemplo Prático

Código C#

```

var anoFiltro = 2018;
var estatisticas = await _context.Database
    .SqlQuery<EstatisticaDTO>($"SELECT Modelo, COUNT(1) as Total FROM Veiculos WHERE
    Ano > {anoFiltro} GROUP BY Modelo")
    .ToListAsync();

```

Atenção: O EF8 sanitiza automaticamente as variáveis passadas via interpolação ("{}") no *SqlQuery*, blindando a sua aplicação contra ataques de *SQL Injection*.

2. Laboratório Prático: O Grande Fechamento de Mês

Cenário: A concessionária precisa rodar uma rotina de fim de mês pesada. O gerente quer aplicar um desconto nos veículos encalhados e, logo em seguida, emitir um relatório achatado dos modelos atualizados.

Missão 1: Atualização em Massa (Commands) Crie um método na sua Infrastructure (dentro de uma classe de repositório ou serviço) que receba o nome de uma Marca. O método deve usar o *.ExecuteUpdateAsync()* para reduzir em 5% o valor de todos os veículos que pertencem a essa marca e que sejam do Ano **2020 ou inferior**. *Dica:* Para acessar o veículo a partir da marca na instrução de update, você pode precisar fazer o *.Where* a partir do *_context.Veiculos* navegando pelas propriedades de relacionamento (*v.Modelo.Marca.Nome == marcaRecebida*).

Missão 2: O Relatório Achatado (Queries) Crie uma Query que retorne uma lista de VeiculoRelatorioDTO (Marca, Modelo, Ano, PrecoAtualizado). Utilize o `_context.Marcas`, filtre pela marca que sofreu o desconto, e use o `.SelectMany()` para achatá-la (Marca -> Modelos -> Veículos), retornando apenas a lista plana de DTOs dos veículos afetados.

3. Paginação Extrema e Controle de Concorrência

Nesta etapa, aprendemos a lidar com os dois maiores pesadelos de sistemas em produção: o travamento do banco ao listar muitos registros e a perda silenciosa de dados quando dois usuários editam a mesma informação simultaneamente.

O Problema do `.Skip()` e `.Take()` (Offset Pagination)

Achar que fazer `.Skip(100000).Take(50)` é rápido é uma ilusão. O comando OFFSET no banco de dados obriga o motor a ler, contar e descartar as primeiras 100.000 linhas apenas para te entregar as próximas 50. Quanto mais longe for a página, mais lenta a API fica.

Atenção (A Realidade do Mercado): Apesar desse gargalo de performance em bases gigantes, a **Paginação Offset ainda é muito utilizada no mercado!** Ela é o padrão absoluto para painéis administrativos (DataGrids) onde o usuário precisa de botões para pular livremente entre as páginas (Ex: [Primeira] [1] [2] [3] [Última]) e a base de dados (ou o resultado do filtro) tem um tamanho controlado.

Como funciona o Offset na Prática: O Front-end envia o número da página desejada e a API faz a matemática. Para o Front desenhar os botões, a API é obrigada a rodar um pesado `Count()` no banco.

C#

// Exemplo de Paginação Offset (Muito usada em Grids Administrativos)

```
public async Task<PaginacaoOffsetResponse<VeiculoRelatorioDTO>>  
ObterRelatorioOffsetAsync(int numeroPagina, int tamanhoPagina)
```

```
{
```

```
    var query = _context.Veiculos.AsNoTracking();
```

```
    // 1. O Custo do Offset: Contar tudo para saber o total de páginas
```

```
    var totalRegistros = await query.CountAsync();
```

```
    // 2. O Cálculo Matemático: (Página Atual - 1) * Tamanho da Página
```

```
    var pular = (numeroPagina - 1) * tamanhoPagina;
```

```
// 3. A Busca Paginada

var veiculosBd = await query

    .OrderBy(v => v.Id)

    .Skip(pular)

    .Take(tamanhoPagina)

    .Select(v => new VeiculoRelatorioDTO { Placa = v.Placa, Preco = v.Preco })

    .ToListAsync();

return new PaginacaoOffsetResponse<VeiculoRelatorioDTO>
{
    Dados = veiculosBd,
    PaginaAtual = numeroPagina,
    TotalRegistros = totalRegistros,
    TotalPaginas = (int)Math.Ceiling(totalRegistros / (double)tamanhoPagina)
};
}
```

A Solução para Alta Performance: Paginação por Chave (Keyset Pagination)

Para relatórios massivos, *feeds* de redes sociais e *Infinite Scroll* (rolagem infinita no mobile), o Offset quebra o banco.

A solução é a Paginação Keyset. Em vez de pular páginas numeradas (1, 2, 3...), nós usamos o último valor conhecido (o Cursor). Dizemos ao banco: *"Traga os próximos 50 carros cujo ID seja MAIOR que o ID 1500"*. Como essa abordagem utiliza os Índices nativos da tabela (B-Tree), o tempo de resposta é quase zero ($O(1)$), não importa o tamanho da base.

💡 **A Analogia do Arquiteto:** > Usar Keyset é como ler um livro usando um marcador de página: você abre instantaneamente onde parou. Usar Offset é como ler um livro sem marcador: se você quer ler a página 300, você é obrigado a folhear (e contar) da página 1 até a 299 toda vez que abrir o livro.

Como funciona o Keyset na Prática: O Front-end envia o Id do último item que ele tem na tela. A API busca a partir desse Id e devolve os dados junto com um novo "Cursor" para a próxima busca.

C#

// Exemplo de Paginação Keyset (Padrão para Infinite Scroll e Alta Performance)

```
public async Task<PaginacaoResponse<VeiculoRelatorioDTO>>
```

```
ObterRelatorioKeysetAsync(int? cursorUltimoId, int tamanhoPagina)
```

```
{
```

```
    var query = _context.Veiculos.AsNoTracking();
```

```
    // O banco pula instantaneamente para esse ponto usando o Índice B-Tree
```

```
    if (cursorUltimoId.HasValue)
```

```
        query = query.Where(v => v.Id > cursorUltimoId.Value);
```

```
    // Buscamos 1 item a mais para saber se tem próxima página sem usar o pesado Count()
```

```
    var veiculosBd = await query
```

```
        .OrderBy(v => v.Id)
```

```
        .Take(tamanhoPagina + 1)
```

```
        .Select(v => new VeiculoRelatorioDTO { Id = v.Id, Placa = v.Placa, Preco = v.Preco })
```

```
        .ToListAsync();
```

```
    var response = new PaginacaoResponse<VeiculoRelatorioDTO>();
```

```
    response.TemMaisPaginas = veiculosBd.Count > tamanhoPagina;
```

```
    if (response.TemMaisPaginas)
```

```
        veiculosBd.RemoveAt(veiculosBd.Count - 1); // Remove o item extra de teste
```

```
    response.Dados = veiculosBd;
```

```

        response.ProximoCursor = veiculosBd.LastOrDefault()?.Id; // O novo bastão para o
Front-end

        return response;
}

```

O Caos da Concorrência (Race Conditions)

O gerente A e o gerente B abrem a tela de edição do mesmo Peugeot 207 ao mesmo tempo. O gerente A altera o status para "Vendido" e salva. Segundos depois, o gerente B altera o preço e salva. O banco sobrescreve os dados do gerente A com os dados antigos que ainda estavam na tela de B. Os dados foram corrompidos silenciosamente.

A Solução: Concorrência Otimista com EF Core 8

Para evitar isso, configuramos uma propriedade de versão (RowVersion ou [Timestamp]). O EF Core passará a incluir automaticamente essa versão no WHERE do UPDATE. Se o gerente B tentar salvar, a versão no banco já não será a mesma (pois mudou quando A salvou), e o EF disparará a exceção DbUpdateConcurrencyException. Basta capturarmos essa exceção no *Handler* e avisarmos o usuário!

Configurando a Versão da Linha (Fluent API / Clean Architecture) Para não poluir nossa classe de Domínio com atributos do framework, configuramos isso diretamente no mapeamento da Infraestrutura.

1. Adicione a propriedade limpa na Entidade:

Código C#

```

// Em ITB.Domain/Entities/Veiculo.cs
public class Veiculo : Entity
{
    public string Placa { get; set; }
    public decimal Preco { get; set; }
    // No Postgres, a versão da linha é mapeada como um inteiro sem sinal (uint)
    public uint VersaoLinha { get; set; }
}

```

2. Mapeie na Infraestrutura usando `.IsRowVersion()`:

Código C#


```
// Em ITB.Infrastructure/Mappings/VeiculoMapping.cs
public void Configure(EntityTypeBuilder<Veiculo> builder)
{
    // O EF criará a coluna no banco e controlará a atualização automaticamente!
    builder.Property(v => v.VersaoLinha)
        .IsRowVersion();
}
```

(Lembre-se: Após adicionar essa configuração, é obrigatório gerar uma nova Migration!)

Tratando o Erro no Handler:

Código C#

```
public async Task<CommandResult> Handle(AtualizarVeiculoCommand comando)
{
    var veiculo = await _context.Veiculos.FindAsync(comando.Id);
    // ... atualiza as propriedades ...

    try
    {
        await _uow.CommitAsync();
        return new CommandResult(true, "Veículo atualizado!");
    }
    catch (DbUpdateConcurrencyException)
    {
        // O pulo do gato do Arquiteto: Interceptamos a colisão!
        return new CommandResult(false, "Este veículo foi modificado por outro usuário enquanto você o editava. Recarregue a página.");
    }
}
```

🔗 Laboratório Prático: Catálogo de Alta Performance e Concorrência

Neste laboratório, vamos colocar à prova as técnicas de paginação e blindagem de dados contra concorrência que acabamos de aprender.

Missão 1: O Catálogo do Feirão (O Duelo das Paginações) Para sentirmos a diferença real de arquitetura na prática, crie duas consultas diferentes na sua camada de Infraestrutura para listar os veículos disponíveis no feirão em pacotes de 20 em 20:

- **Passo A (A Tradicional):** Crie a query `ObterVeiculosOffsetAsync(int numeroPagina, int tamanhoPagina)`. Implemente a paginação administrativa usando a matemática de deslocamento com o `.Skip()` e o `.Take()`.
- **Passo B (A de Alta Performance):** Crie a query `ObterVeiculosKeysetAsync(int? ultimoIdDaPagina, int tamanhoPagina)`. Implemente a paginação focada em performance usando a lógica de cursor com o `.Where(v => v.Id > ultimoIdDaPagina)` e o `.Take()`.

Missão 2: A Atualização Competitiva (Concorrência)

1. Na Entidade `Veiculo`, adicione a propriedade `VersaoLinha` (do tipo `byte[]`) e configure-a como `IsRowVersion` na Fluent API (lembre-se de rodar a Migration para criar a coluna no banco!).
2. Simule o tratamento de colisão de dados capturando a exceção `DbUpdateConcurrencyException` dentro da classe `AtualizarVeiculoHandler`. Garanta que, se dois usuários tentarem editar o mesmo carro ao mesmo tempo, o segundo a salvar receba uma mensagem de erro amigável na API, impedindo a perda silenciosa de dados.